Lab 14 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — Van lab 08 naar het cluster

Antwoord. `ENV` → blijft in de image (een standaardwaarde, ongewijzigd). `-e` → `env:` met een vaste `value` in het template. `--env-file api.env` → een ConfigMap (`--from-env-file`) ingespoten via `envFrom`. `application.yaml` als bind mount → een ConfigMap `--from-file` gemount als volume in `/config/`. `podman secret` + `_FILE` → een Secret gemount als volume `secret:` met `DB_PASSWORD_FILE`. Zonder equivalent: de `-e` **getypt op de commandoregel** — en dat is een goede zaak: in het cluster is elke configuratie een geversioneerd en herleesbaar object, geen argument dat verloren gaat in de shellgeschiedenis.

Waarom. Het cluster heeft geen commandoregel: elke waarde moet in de API bestaan om op gelijk welke node toegepast te worden.

Nuance. `kubectl set env deployment/api APP_PROFILE=prod` bestaat en wijzigt het template rechtstreeks — handig voor een test, te vermijden in productie (de waarde staat niet meer in Git).

Voorbeeld.

```
kubectl create configmap api-config --from-env-file=api.env --dry-run=client -o yaml
```

Vraag 2 — Bevroren variabelen, levende bestanden

Antwoord. Eerste `grep`: `APP_MESSAGE=bonjour depuis une ConfigMap`, **de oude waarde** — variabelen worden bij het opstarten in het proces gekopieerd. Tweede `cat` (na een minuut): `<h1>modifie</h1>` — het volume wordt door de kubelet bijgewerkt. Opdat de API het nieuwe bericht ziet: `kubectl rollout restart deployment/api`, dat de Pods opnieuw aanmaakt.

Waarom. Een omgevingsvariabele hoort bij het proces; een ConfigMap-volume is een map die de kubelet periodiek opnieuw synchroniseert (symbolische link `..data` atomair omgeschakeld).

Nuance. De bijwerkingstijd van het volume hangt af van de synchronisatieperiode van de kubelet (standaard één minuut) en van de cache; reken op een tot twee minuten. En nginx zelf heeft `default.conf` bij het opstarten geladen: een nieuw configuratiebestand vereist een `nginx -s reload`, een nieuwe `index.html` wordt onmiddellijk geserveerd.

Voorbeeld.

```
kubectl exec deploy/web -- ls -l /usr/share/nginx/html/     # index.html -> ..data/index.html
kubectl rollout restart deployment/api && kubectl exec deploy/api -- env | grep APP_MESSAGE
```

Vraag 3 — `CreateContainerConfigError`

Antwoord. `kubectl describe pod api-5dff955d5b-rz7sd` toont in de Events: `Error: couldn't find key profil in ConfigMap lab14/api-config` — de gerefereerde sleutel bestaat niet (`profil` in plaats van `APP_PROFILE`). De kubelet kan de omgeving van de container niet opbouwen en start hem dus niet. De oude Pod draait nog omdat de rollout wacht tot de nieuwe klaar is voor hij de

oude weghaalt (lab 12): de dienst wordt niet geraakt. Correctie: de sleutel herstellen (of `rollout undo`).

Waarom. Verwijzingen naar ConfigMaps en Secrets worden **bij het aanmaken van de container** opgelost, niet bij de `apply`: een manifest kan aanvaard worden en later falen.

Nuance. `optional: true` op de verwijzing (`configMapKeyRef` of `configMapRef`) laat een afwezige sleutel of object negeren in plaats van te blokkeren — alleen te gebruiken voor echt facultatieve configuratie.

Voorbeeld.

```
kubectl describe pod -l app=api | grep -A1 "couldn't find key"
kubectl rollout undo deployment/api
```

Vraag 4 — "Versleuteld, dus veilig"

Antwoord. `kubectl get secret db-secret -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d` toont `S3cr3t-pg`: base64 is een **codering**, omkeerbaar door iedereen. Wat een Secret werkelijk toevoegt: (1) een **aparte objectklasse**, waarvan RBAC het lezen apart van de ConfigMaps kan verbieden; (2) een mount in **tmpfs** op de node, nooit op zijn schijf geschreven; (3) het **maskeren** in `describe` en de tools, wat lekken via kopiëren-plakken vermijdt. De drie bedrijfsmaatregelen: versleuteling in rust in etcd (`EncryptionConfiguration` of KMS), strikte RBAC op `get`/`list` `secrets`, en geen enkel Secret in klare tekst in Git (External Secrets, Sealed Secrets, SOPS).

Waarom. Base64 bestaat alleen om willekeurige bytes in JSON/YAML te vervoeren; het heeft nooit een beschermende rol gehad.

Nuance. Zelfs versleuteld in rust is een Secret **in klare tekst** voor wie het via de API mag lezen: RBAC is de echte grens.

Voorbeeld.

```
kubectl get secret db-secret -o yaml | grep POSTGRES_PASSWORD     # UzNjcjN0LXBn
echo UzNjcjN0LXBn | base64 -d                                      # S3cr3t-pg
```

Vraag 5 — Variabele of bestand voor het wachtwoord

Antwoord. Zichtbaarheid: als variabele toont `kubectl exec deploy/api -- env` het wachtwoord (en elke *dump* van de omgeving ook); als bestand is alleen het pad zichtbaar, de waarde vereist het bestand te openen. Rotatie: als variabele wordt een nieuw Secret pas gezien na `rollout restart`; als bestand werkt de kubelet het bestand in een minuut bij, en volgt een applicatie die het bestand opnieuw leest (of een pool die met het nieuwe wachtwoord opnieuw verbindt) zonder herstart. Spring Boot: beide werken, de tweede via `spring.config.import=optional:configtree:/run/secrets/db/`. Men kiest het **bestand**, zoals in lab 08.

Waarom. De omgeving is een ruim blootgestelde metadata; een gemount bestand is alleen toegankelijk voor het proces.

Nuance. Veel officiële images aanvaarden alleen de variabele (`POSTGRES_PASSWORD`) **of** het bestand (`POSTGRES_PASSWORD_FILE`): raadpleeg de documentatie van de image.

Voorbeeld.

```
kubectl exec deploy/api -- env | grep -c S3cr3t        # 0 met het volume
kubectl exec deploy/api -- cat /run/secrets/db/POSTGRES_PASSWORD
```

Vraag 6 — `subPath` wordt niet bijgewerkt

Antwoord. Met `subPath` **kopieert** de kubelet het bestand bij het opstarten in de container in plaats van de map met symbolische links te mounten: wijzigingen van de ConfigMap worden nooit doorgegeven. Twee uitwegen: (1) de ConfigMap op een **eigen map** mounten (`/etc/nginx/conf.d/` volledig, of `/etc/nginx/extra/` geïncludeerd vanuit de hoofdconfiguratie) zonder `subPath`; (2) de `subPath` aanvaarden en bij elke wijziging een `rollout restart` uitlokken — of de naam van de ConfigMap veranderen (vraag 12), wat het template wijzigt en de rollout uitlokt.

Waarom. `subPath` bestaat om één bestand in een map te injecteren die andere dingen bevat; de prijs is het verlies van de dynamische bijwerking.

Nuance. Dezelfde valkuil geldt voor Secrets gemount met `subPath`: de rotatie is niet zichtbaar.

Voorbeeld.

```
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.containers[0].volumeMounts}' | grep -o 'subPath[^}]*'
```

Vraag 7 — Hetzelfde Deployment, andere namespace

Antwoord. Het Deployment wordt aangemaakt in `prod`, zijn Pods starten met `APP_PROFILE=prod` en het productiewachtwoord: de verwijzingen worden **in de namespace van de Pod** opgelost, naar de objecten `api-config` en `db-secret` die daar staan. Niets in het manifest is veranderd. Dat toont aan dat het promoveren van een image (lab 10) een `apply` van hetzelfde bestand is in een andere namespace (of cluster), waar de configuratie anders is: het geteste artefact is het uitgerolde artefact.

Waarom. Verwijzingen op naam zijn lokaal aan de namespace; dat is de grens die de configuratie draagt.

Nuance. Ontbreekt `api-config` in `prod`, dan blijven de Pods in `CreateContainerConfigError` (vraag 3): de ConfigMap maakt deel uit van wat uitgerold moet worden, en Helm/Kustomize (lab 17) nemen ze mee met de rest.

Voorbeeld.

```
kubectl apply -f deployment-api.yaml -n prod && kubectl exec -n prod deploy/api -- env | grep APP_PROFILE   # prod
```

Vraag 8 — `--dry-run=client -o yaml`

Antwoord. Het commando **stuurt niets** naar het cluster: het drukt het volledige `Secret`-manifest af, met `data.password` al in base64 gecodeerd (`eA==` voor `x`), klaar om in een bestand bewaard te worden. Je gebruikt het om de exacte structuur te krijgen zonder je te vergissen in codering of inspringing. Wat je niet mag doen: dat bestand zo committen — het bevat het secret, base64 of niet. Het moet versleuteld worden (SOPS, Sealed Secrets) voor het in Git gaat, of als model zonder waarde dienen.

Waarom. `--dry-run=client` laat `kubectl` al het werk doen om het object op te bouwen, zonder aanroep naar de API.

Nuance. `--dry-run=server` stuurt het object ter validatie zonder het te bewaren: nuttig om de webhooks en de regels van het cluster te testen.

Voorbeeld.

```
kubectl create secret generic db --from-literal=password=x --dry-run=client -o yaml | grep -A1
'^data:'   # password: eA==
```

Vraag 9 — Secret in een ticket geplakt

Antwoord. (1) Het Secret is **gecompromitteerd**: base64 of niet, het is gelezen. (2) **Rotatie**: nieuw wachtwoord aan de databasekant, `kubectl apply` (of de secretsbeheerder) van een bijgewerkte `db-secret`. (3) **Verspreiding**: is het Secret als volume gemount, dan zien de Pods het nieuwe bestand binnen een minuut, en volgt een applicatie die het bestand opnieuw leest; als variabele is `rollout restart deployment/api` verplicht. (4) Intrekking van het oude wachtwoord, controle van de databaselogs, opschonen van het ticket als laatste. Wat de rotatie snel maakt: een Secret gerefereerd op naam, gemount als bestand, gelezen door een applicatie die opnieuw kan verbinden; traag: een waarde overgenomen in meerdere manifesten, als variabele, met Pods die een voor een herstart moeten worden.

Waarom. Het enige veilige antwoord blijft de gelekte waarde waardeloos maken.

Nuance. De RBAC van lab 16 beperkt het aantal mensen dat zo'n `get -o yaml` kan produceren: dat is de preventieve maatregel.

Voorbeeld.

```
kubectl create secret generic db-secret --from-literal=POSTGRES_PASSWORD=nieuw --dry-run=client
-o yaml | kubectl apply -f -
kubectl rollout restart deployment/api
```

Vraag 10 — Een lege map

Antwoord. De **`volumeMounts`** ontbreekt (of zijn `mountPath` wijst niet naar `/usr/share/nginx/html`): de ConfigMap is misschien gedeclareerd onder `volumes:`, maar niets mount ze in de container — of het volume is op een ander pad gemount. Controle: `kubectl get pod -l app=web -o yaml` en `spec.volumes` (het volume `html` met `configMap: web-html`) vergelijken met `spec.containers[0].volumeMounts` (een mount `html` op `/usr/share/nginx/html`). Een lege map (en niet de oorspronkelijke `index.html` van de image) wijst erop dat er **wel** een volume gemount is, maar niet het juiste: bijvoorbeeld een lege ConfigMap of een met een andere naam.

Waarom. `volumes` declareert, `volumeMounts` sluit aan; beide zijn nodig, en een mount verbergt wat de image bevatte.

Nuance. `kubectl describe pod` toont `Mounts:` en `Volumes:` leesbaar, en een volume waarvan de ConfigMap niet bestaat, blokkeert in `ContainerCreating` met `configmap "web-html" not found`.

Voorbeeld.

```
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.containers[0].volumeMounts}' ; echo
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.volumes}' ; echo
```

Vraag 11 — Heel `/etc/nginx/conf.d` mounten

Antwoord. De hele map mounten **verbergt** de inhoud van de image op die plaats: de oorspronkelijke `default.conf` verdwijnt, vervangen door het enige bestand van de ConfigMap — dat is gewild, aangezien het dat bestand is dat we vervangen, en het behoudt de dynamische bijwerking (geen `subPath`). Om een tweede `.conf`-bestand toe te voegen: het in **dezelfde** ConfigMap zetten (een sleutel meer, een bestand meer in de map), of `projected` gebruiken om meerdere ConfigMaps in één mount te combineren.

Waarom. Een mount is een vervanging van een map, precies zoals een bind mount in lab 06; de image eronder wordt niet samengevoegd.

Nuance. Verwachtte de image andere bestanden in die map (hier niet), dan zou je ze in de ConfigMap moeten overnemen of elders mounten.

Voorbeeld.

```
kubectl exec deploy/web -- ls /etc/nginx/conf.d/     # alleen default.conf, uit de ConfigMap
```

Vraag 12 — `immutable` of geversioneerde naam

Antwoord. `immutable: true` verbiedt elke wijziging van het object: bescherming tegen een toevallige bewerking in productie, en de kubelet houdt op het te bewaken (minder belasting voor de API server) — maar de configuratie veranderen vereist **een ander object aan te maken** en de verwijzing te veranderen. `api-config-v2` noemen en het Deployment ernaar laten wijzen, verandert het **template**, dus lokt een **rollout** uit: de Pods herstarten met de nieuwe configuratie, en `rollout undo` brengt de oude terug (de oude ReplicaSet verwijst naar `api-config-v1`). Daarom voegen Kustomize (`configMapGenerator`) en Helm (checksum-annotatie) een hash-achtervoegsel toe: elke wijziging van de configuratie wordt een revisie van het Deployment, traceerbaar en omkeerbaar.

Waarom. Een ter plekke gewijzigde ConfigMap is geen revisie; een hernoemde ConfigMap wel.

Nuance. De oude ConfigMaps met achtervoegsel stapelen zich op; Kustomize en Helm ruimen ze op (of niet) naargelang de configuratie.

Voorbeeld.

```
kubectl patch configmap api-config -p '{"immutable":true}' && kubectl patch configmap api-config -p '{"data":{"APP_PROFILE":"x"}}'   # field is immutable
```

Vraag 13 — Geen enkel Secret in Git

Antwoord. De repository bevat het Deployment (dat naar `db-secret` verwijst op naam) en een object **ExternalSecret**: "maak een Secret `db-secret` aan met de sleutel `POSTGRES_PASSWORD` uit Vault op het pad `secret/app/prod/db`". Geen enkele waarde. In het cluster leest de **External Secrets Operator** dat object, authenticeert zich bij de secretsbeheerder (Vault, AWS Secrets Manager…), haalt de waarde op en **maakt of werkt** het Kubernetes-Secret bij — en vernieuwt het periodiek, wat de rotatie doet. Met Sealed Secrets bevat de repository het Secret **versleuteld**

met de publieke sleutel van het cluster, en ontsleutelt een controller het tot een gewoon Secret. Het verband met lab 08: de secretsbeheerder is de bron van waarheid, het Kubernetes-Secret is maar een lokale cache die in de Pods ingespoten wordt.

Waarom. Git is een verspreidingsmedium; het cluster is de plaats waar een secret onder toegangscontrole kan leven.

Nuance. Beide benaderingen houden het Secret **in klare tekst in de API** eens het aangemaakt is: versleuteling in rust en RBAC blijven nodig.

Voorbeeld.

```
kind: ExternalSecret
spec:
  secretStoreRef: { name: vault }
  target: { name: db-secret }
  data:
    - secretKey: POSTGRES_PASSWORD
      remoteRef: { key: secret/app/prod/db, property: password }
```